Convoca'm: revisión de calidad y seguridad

Valoración del proyecto, hallazgos de seguridad y propuesta de mejoras «sin volvernos locos». 27 de octubre de 2026 · rama master (commit d721fba).

Resumen

Para ser un TFM, el proyecto está **por encima de la media**, sobre todo en arquitectura y en cómo se protegen los datos de cada usuario. Los fallos de seguridad encontrados son de **configuración y de detalle, no de diseño**: ninguno permite a un usuario leer o modificar datos de otro.

Lo más urgente son dos cosas ajenas al código de la aplicación: la base de datos está accesible desde la red local con una contraseña trivial, y hay dependencias con vulnerabilidades conocidas. Además, en el dashboard se quedó activado un modo de pruebas que cambia el comportamiento especificado en SPEC-009.

Qué se ha comprobado

- Lectura del código de autenticación, todas las Server Actions, el endpoint interno de CIDO, las consultas SQL crudas, el detalle y el buscador de convocatorias, y la configuración (Next.js, Docker, .gitignore).
- Tests unitarios (`pnpm test:run`): **46 archivos, 355 tests, todos en verde**.
- Tipos (`tsc --noEmit`) y lint (`eslint`): **sin errores**.
- Auditoría de dependencias de producción (`pnpm audit --prod`): 3 críticas, 13 altas, 20 moderadas, 4 bajas.
- Búsqueda de secretos en todo el historial de git: no se ha subido nunca ningún .env ni credenciales reales.
- Puertos abiertos en la máquina: PostgreSQL escucha en 0.0.0.0:5434.
- **Pendiente:** las pruebas de ataque contra el servidor en marcha no pudieron ejecutarse en la sesión (ver el último apartado). Los puntos marcados como «por lectura de código» no se han reproducido en vivo.

Puntos fuertes

- **Autorización bien hecha.** Todas las Server Actions obtienen el usuario de la sesión (`auth.api.getSession`) y nunca del formulario. Las titulaciones se buscan, actualizan y borran filtrando a la vez por `id` y `userProfileId`; los seguimientos van por `(userId, selectionProcessId)`. No se ha encontrado ninguna forma de tocar datos ajenos (IDOR).
- **SQL sin inyección.** La búsqueda tolerante usa `Prisma.sql` con parámetros; no hay `$queryRawUnsafe` ni concatenación de cadenas.
- **Endpoint interno sólido.** Si el token falta o no es válido, se deniega siempre; la comparación es en tiempo constante (hash SHA-256 + timingSafeEqual), se valida la longitud y tiene tests.
- `returnTo` **del detalle con lista blanca de rutas**: no se puede usar para redirigir fuera.
- **Validación de entrada** en las actions: regex para IDs, enums en lista blanca, máximo de 2000 territorios. Los mensajes de error que ve el usuario están en lista blanca y no filtran detalles de Prisma.
- **Calidad base:** tests, tipos y lint limpios; secretos con buena longitud y fuera de git.

- **Arquitectura hexagonal coherente**, con composition roots, entidades con create/restore y repositorios en memoria para los tests.
- **Sincronización con CIDO bien pensada:** borrado lógico (sourceActive), reconciliación de convocatorias desaparecidas y hashes de origen.
- Detalles cuidados: zona horaria Europe/Madrid, enlaces externos con `rel='noreferrer'`, React 19 bloquea URLs javascript: en href.

Hallazgos de seguridad, por prioridad

El esfuerzo es orientativo. «Por lectura de código» indica que la conclusión no se ha reproducido contra el servidor.

#	Hallazgo	Gravedad	Esfuerzo
1	**PostgreSQL abierto a toda la red local.** docker-compose.yml publica 0.0.0.0:5434 con usuario/clave convocam/convocam (comprobado). Trabajas desde varias redes (192.168.0.x, 3.x, 2.x): cualquiera en la misma wifi puede entrar y ver hashes de contraseñas y tokens de sesión.	**Alta**	1 línea
2	**Dependencias vulnerables:** pnpm audit --prod da 3 críticas, 13 altas y 20 moderadas. No se pudo desglosar qué paquetes son los críticos.	**Alta**	Actualizar y reauditar
3	**Redirección abierta en el login.** login-form.tsx solo comprueba que returnTo empiece por «/» y no por «//». Con ?returnTo=/\evil.com pasa el filtro y el navegador lo interpreta como https://evil.com. Útil para phishing con un enlace legítimo de tu dominio.	**Media**	Pocas líneas
4	**Política de contraseñas solo en el cliente.** better-auth por defecto exige 8 caracteres; llamando a la API directamente se acepta «aaaaaaaa». Por lectura de código.	**Media**	Config. en auth.ts
5	**Sin cabeceras de seguridad** (X-Frame-Options, nosniff, Referrer-Policy). next.config.ts no define headers(). Sin ellas se pueden meter tus páginas en un iframe y engañar al usuario para que pulse botones (clickjacking).	**Media-baja**	Un bloque headers()
6	**Búsqueda anónima sin límites.** Cada palabra de q añade unas 20 llamadas a word_similarity y, con orden por relevancia, se cargan en memoria todas las filas coincidentes. Una URL con miles de palabras es cara para la BD.	**Baja-media**	Limitar q
7	**IDs muy grandes acaban en error 500.** Number.isInteger(1e20) es true, así que /convocatorias/1e300 o ?scopeId=1e20 llegan a Prisma y fallan como Int fuera de rango. Por lectura de código.	**Baja**	Helper común
8	**Textos libres sin longitud máxima** (nombre, apellidos, titulación).	**Baja**	Validar en dominio
9	**Orígenes de la LAN fijos en el código.** trustedOrigins y allowedDevOrigins tienen IPs de tu red escritas a mano; en producción seguirían siendo de confianza.	**Baja**	Leer de .env
10	**Enumeración de usuarios en el registro.** Registrar un email existente responde que ya existe. Habitual y aceptable en un TFM; solo para conocerlo.	**Baja**	Aceptar

Lo que no haría, para no volvernos locos

Una CSP estricta con nonces (choca con MapLibre y con los scripts de Next.js), 2FA, verificación de email o un WAF. Solo tendría sentido si la aplicación se despliega en público con usuarios reales. El límite de intentos de login de better-auth ya viene activado en producción; en desarrollo está desactivado, y no hace falta tocarlo salvo que se despliegue con varias instancias (entonces, almacenamiento en base de datos).

Puntos débiles que no son de seguridad

- `TEST_RECENT_PUBLICATIONS = true` se quedó activado en src/app/dashboard/page.tsx:61 (marcado «SOLO PARA PRUEBAS»). El dashboard enseña publicaciones anteriores al inicio del seguimiento, contradice SPEC-009 y consulta prisma directamente desde la página,

saltándose la arquitectura. **Es lo primero que vería un tribunal.**

- `revalidatePath` **apunta a una ruta que no existe** en src/app/convocatorias/[id]/actions.ts (`/mis-convocatorias/${id}`). Probablemente funciona de rebote porque cualquier revalidatePath vacía la caché del navegador.
- **console.log de depuración** en src/app/mis-convocatorias/actions.ts:63.
- **«He olvidado la contraseña»** enlaza a /forgot-password, que no existe (404).
- **Archivos enormes y código duplicado:** convocatorias/[id]/page.tsx (1917 líneas), el repositorio de consultas (1650), convocatorias/page.tsx (1476); getCataloniaTodayUtc y resolveSelectionProcessSort repetidos. Sin Prettier: hay JSX con una palabra por línea.
- **Errores identificados por el texto del mensaje** (`error.message === '...'`): frágil ante cualquier cambio de redacción; mejor clases de error propias del dominio.
- **Tests de integración solo en user-profile.** El SQL crudo del buscador y del dashboard no se prueba contra PostgreSQL real.
- **Llamadas a CIDO sin timeout:** una respuesta colgada bloquea la sincronización completa.
- **Seguir una convocatoria inexistente o hacer doble clic** produce un error 500 (clave foránea o unicidad) en vez de un mensaje controlado.
- **docs/TO DO.md** está en .gitignore pero sigue versionado.

Propuesta mínima (unas 2–3 horas)

Ordenada por relación impacto/esfuerzo. Los fragmentos son propuestas para revisar, no cambios aplicados: no se ha modificado ningún archivo del proyecto.

1. Atar PostgreSQL a localhost (hallazgo 1)

En docker-compose.yml. Después: docker compose up -d. DATABASE_URL no cambia.

```
    ports:
      - "127.0.0.1:5434:5432"
```

2. Desactivar el modo de pruebas del dashboard

En src/app/dashboard/page.tsx. Mejor aún: borrar la constante, loadRecentPublicationsForTest y el import de prisma, dejando solo el caso de uso.

```
const TEST_RECENT_PUBLICATIONS = false;
```

3. Actualizar dependencias (hallazgo 2)

Revisar qué paquetes son los críticos y actualizar; volver a pasar tests, tipos y lint.

```
pnpm audit --prod
pnpm update next eslint-config-next better-auth @better-auth/prisma-adapter
pnpm update --latest   # opcional, revisando el changelog
pnpm audit --prod && pnpm test:run && pnpm exec tsc --noEmit
```

4. Corregir el returnTo del login (hallazgo 3)

En src/components/auth/login-form.tsx: resolver la URL igual que el navegador y exigir el mismo origen.

```
function getSafeReturnTo(value: string | null): string {
  if (!value) return "/dashboard";
  try {
    const url = new URL(value, window.location.origin);
    if (url.origin !== window.location.origin) return "/dashboard";
    return url.pathname + url.search + url.hash;
  } catch {
    return "/dashboard";
  }
}

// ...
router.replace(getSafeReturnTo(searchParams.get("returnTo")));
```

5. Política de contraseñas en el servidor (hallazgo 4)

En src/lib/auth.ts. Longitudes nativas de better-auth; la regla «letra y número» con un hook. Verificar los nombres de la API con la versión instalada antes de aplicarlo.

```
import { APIError, createAuthMiddleware } from "better-auth/api";

emailAndPassword: {
  enabled: true,
  minPasswordLength: 8,
  maxPasswordLength: 128,
},
hooks: {
  before: createAuthMiddleware(async (ctx) => {
    if (ctx.path !== "/sign-up/email") return;
    const password = String(ctx.body?.password ?? "");
    const name = String(ctx.body?.name ?? "");
    if (!/[A-Za-z]/.test(password) || !/\d/.test(password)) {
      throw new APIError("BAD_REQUEST", {
        message: "La contraseña debe tener al menos una letra y un número.",
      });
    }
    if (name.trim().length === 0 || name.length > 100) {
      throw new APIError("BAD_REQUEST", { message: "El nombre no es válido." });
    }
  }),
},
```

6. Cabeceras de seguridad básicas (hallazgo 5)

En next.config.ts. Sin CSP para no romper el mapa; frame-ancestors equivale a X-Frame-Options.

```
const nextConfig: NextConfig = {
  poweredByHeader: false,
  allowedDevOrigins: [/* ... */],
  async headers() {
    return [{
      source: "/:path*",
      headers: [
        { key: "X-Frame-Options", value: "DENY" },
        { key: "Content-Security-Policy", value: "frame-ancestors 'none'" },
        { key: "X-Content-Type-Options", value: "nosniff" },
        { key: "Referrer-Policy", value: "strict-origin-when-cross-origin" },
        { key: "Permissions-Policy", value: "camera=(), microphone=(), geolocation=()" },
      ],
    }];
  },
};
```

7. Limitar la búsqueda (hallazgo 6)

En la página del buscador y, como defensa en profundidad, en findTextMatches del repositorio.

```
// src/app/convocatorias/page.tsx (y para-mi si aplica)
const query = (params.q?.trim() ?? "").slice(0, 100);

// prisma-selection-process-query-repository.ts, findTextMatches
const terms = text.trim().split(/\s+/).filter(Boolean).slice(0, 8);
```

8. Validación común de IDs y longitudes (hallazgos 7 y 8)

Un helper compartido (p. ej. src/shared/presentation o shared/domain) usado en páginas y actions, y un máximo de longitud en las entidades de dominio.

```
const MAX_INT4 = 2_147_483_647;

export function parsePositiveId(value: unknown): number | null {
  if (typeof value !== "string" || !/^[1-9]\d{0,9}$/.test(value)) return null;
  const id = Number(value);
  return id <= MAX_INT4 ? id : null;
}

// En UserProfile / UserAcademicQualification:
if (normalizedName.length > 150) {
  throw new Error("La denominación es demasiado larga");
}
```

9. Limpiezas rápidas

Sin riesgo y mejoran la impresión del código.

- Corregir `revalidatePath` en convocatorias/[id]/actions.ts a `/convocatorias/${id}` y `/mis-convocatorias`.
- Quitar el console.log de mis-convocatorias/actions.ts.
- Ocultar el enlace «He olvidado la contraseña» hasta que exista la página.
- Mover trustedOrigins y allowedDevOrigins a una variable de entorno (hallazgo 9).
- Añadir `signal: AbortSignal.timeout(30_000)` a los fetch de cido-client.ts.
- Quitar docs/TO DO.md del índice: `git rm --cached 'docs/TO DO.md'`.

Si sobra tiempo (opcional)

- Prettier con formato automático, para normalizar los archivos grandes antes de la defensa.
- Extraer los helpers de fechas duplicados a un único módulo compartido.
- Un par de tests de integración del buscador (SQL crudo) y del dashboard contra PostgreSQL real.
- Clases de error de dominio (p. ej. DomainValidationError) en lugar de comparar mensajes.

Pruebas de ataque pendientes

El script quedó preparado pero no pudo ejecutarse en la sesión. Prueba, contra el servidor de desarrollo: cabeceras de seguridad, endpoint interno sin token, registro saltándose la validación del cliente, nombre de 20.000 caracteres, enumeración de usuarios, CSRF con otro Origin, flags de la cookie de sesión, fuerza bruta, parámetros numéricos extremos, búsqueda abusiva, redirección abierta, returnTo del detalle y páginas privadas sin sesión.

Con el servidor arrancado (pnpm dev), desde Claude Code:

```
! bash /tmp/claude-1000/-home-jholgado-apps-tfm-convocam-app/d8af07b0-f3f9-47c7-b5ee-4db4216bfe5f/scratchpad/sect
```

El script crea usuarios sectest-*@example.test. Para borrarlos (el borrado arrastra sus perfiles):

```
docker exec convocam-postgres psql -U convocam \
  -c "DELETE FROM \"user\" WHERE email LIKE 'sectest-%@example.test';"
```

La prueba de fuerza bruta solo es representativa en modo producción (pnpm build && pnpm start), porque better-auth no limita intentos en desarrollo. El script está en la carpeta temporal de la sesión: si lo quieres conservar, cópialo a otra ubicación.

Nota: al arrancar pnpm dev, Next.js añadió por su cuenta un bloque «nextjs-agent-rules» al final de CLAUDE.md. Decide si conservarlo.